PYTHON ACTIVITY-BASED LEARNING

Documentation for 10 Selected Case Studies

Student Name	Mrudhula Varnakavi
Roll Number	26001163
College Name	Dr RVR NRI (Deemed To Be University)
Branch / Class	CSM_05

Each case includes the problem statement, objectives, assumptions, algorithm, standard flowchart, pseudocode, Python source code, test cases, output screenshot, and conclusion.

Contents

Case	Case Study	Case	Case Study
2	Progressive Electricity Billing	11	Examination Result
3	Hospital Billing	12	Attendance Recovery
4	Employee Payroll	13	ATM Transaction Simulation
7	Retail Billing	27	Numeric Pattern Generator
8	Manufacturing Cost and Profit	39	Multi-Stage Examination Eligibility System

Standard flowchart symbols: oval = Start/End; parallelogram = Input/Output; rectangle = Process; diamond = Decision. Arrows indicate control flow.

Case 2: Progressive Electricity Billing

Problem Statement

Calculate an electricity bill using progressive unit rates and add a fixed charge of 50.

Objectives

Use if-elif-else conditions to apply different rates to unit ranges and calculate the final bill.

Assumptions

Units are entered as a non-negative whole number. First 100 units cost 2 per unit, the next 100 units cost 3 per unit, units above 200 cost 5 per unit, and a fixed charge of 50 is added.

Algorithm

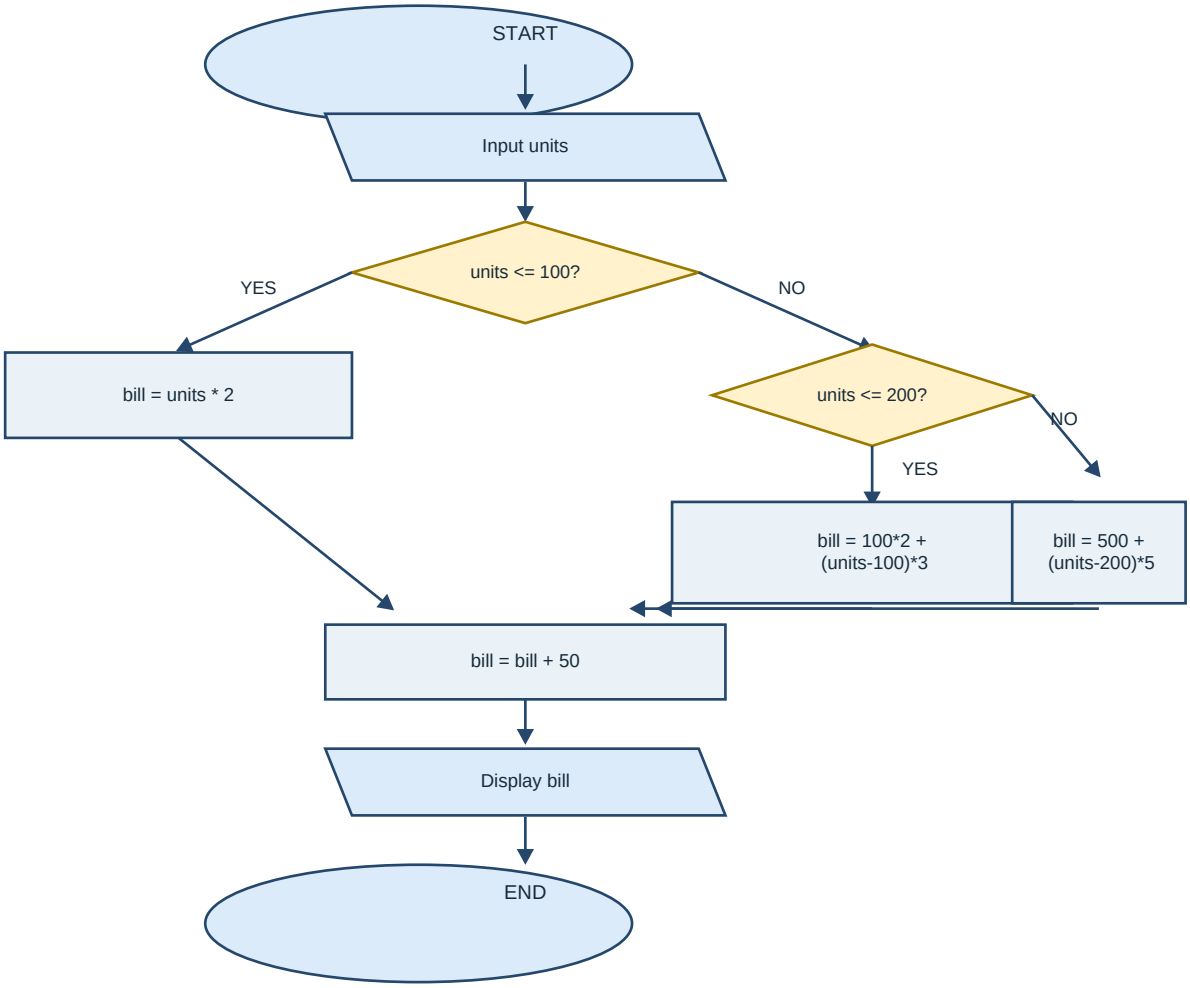
1. Start.
2. Read electricity units.
3. If units are at most 100, calculate units × 2.
4. Otherwise, if units are at most 200, calculate 100 × 2 + (units – 100) × 3.
5. Otherwise, calculate 100 × 2 + 100 × 3 + (units – 200) × 5.
6. Add the fixed charge of 50.
7. Display the electricity bill.
8. Stop.

Pseudocode

```
START
INPUT units
IF units <= 100
    bill = units * 2
ELSE IF units <= 200
    bill = 100 * 2 + (units - 100) * 3
ELSE
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5
END IF
bill = bill + 50
OUTPUT bill
STOP
```

Case 2: Progressive Electricity Billing — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Case 2: Progressive Electricity Billing — Python Source Code

```
units = int(input("Enter electricity units: "))

if units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5

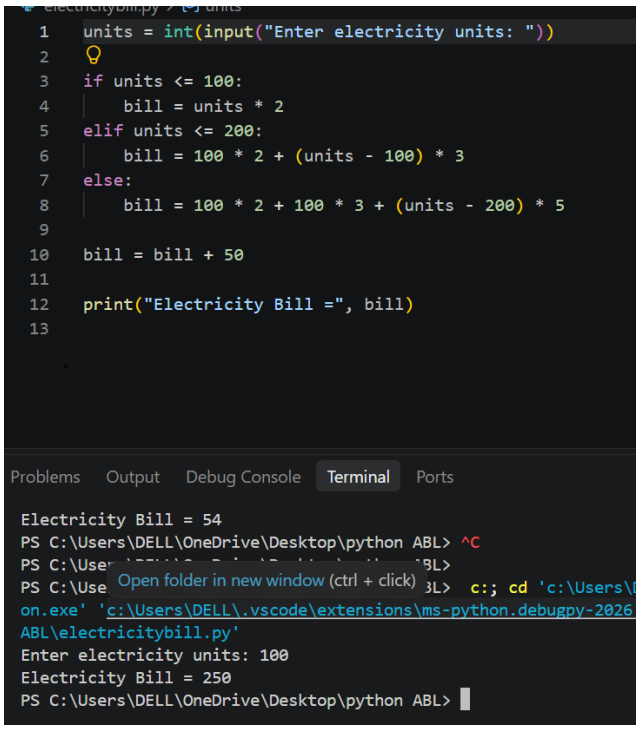
bill = bill + 50
print("Electricity Bill =", bill)
```

Case 2: Progressive Electricity Billing — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
100	250	100 × 2 + 50 = 250
150	400	100 × 2 + 50 × 3 + 50 = 400
250	900	100 × 2 + 100 × 3 + 50 × 5 + 50 = 900

Output Screenshot



Conclusion

The program demonstrates progressive billing using conditional statements and a fixed charge.

Case 3: Hospital Billing

Problem Statement

Calculate the total hospital bill from consultation, diagnostic tests, and treatment charges, then apply a discount when the total is at least 5000.

Objectives

Read three charges, calculate the total, use a condition to decide the discount, and display the final bill.

Assumptions

All charges are non-negative. A 10% discount is applied when the total is 5000 or more; otherwise, no discount is applied.

Algorithm

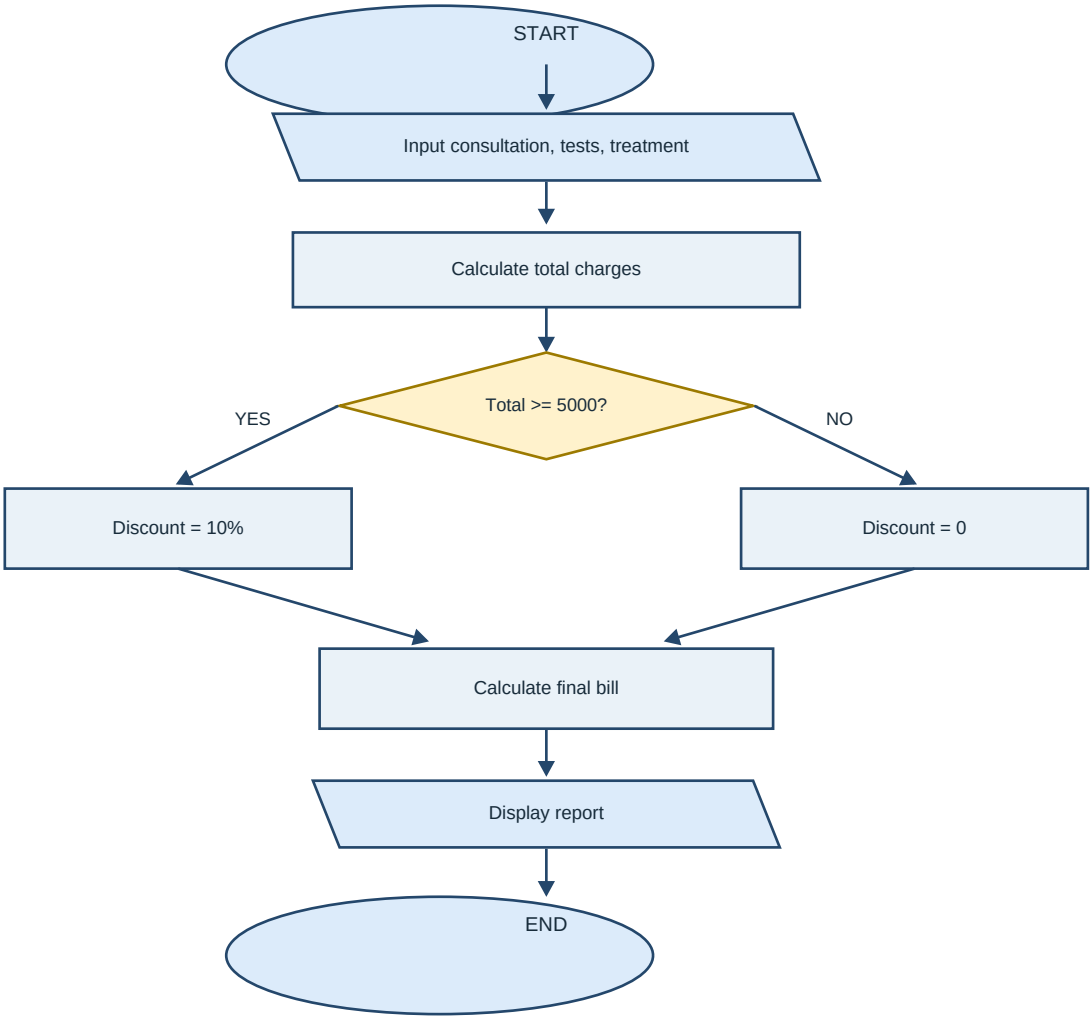
1. Start.
2. Read consultation fee, diagnostic charges, and treatment charges.
3. Add the three charges to get the total.
4. If total is at least 5000, set discount to 10% of total; otherwise set discount to 0.
5. Subtract discount from total.
6. Display total, discount, and final bill.
7. Stop.

Pseudocode

```
START
INPUT consultation, tests, treatment
total = consultation + tests + treatment
IF total >= 5000
    discount = total * 0.10
ELSE
    discount = 0
END IF
final_bill = total - discount
OUTPUT total, discount, final_bill
STOP
```

Case 3: Hospital Billing — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Case 3: Hospital Billing — Python Source Code

```
consultation = float(input("Enter consultation fee: "))
tests = float(input("Enter diagnostic charges: "))
treatment = float(input("Enter treatment charges: "))

total = consultation + tests + treatment

if total >= 5000:
    discount = total * 0.10
else:
    discount = 0

final_bill = total - discount

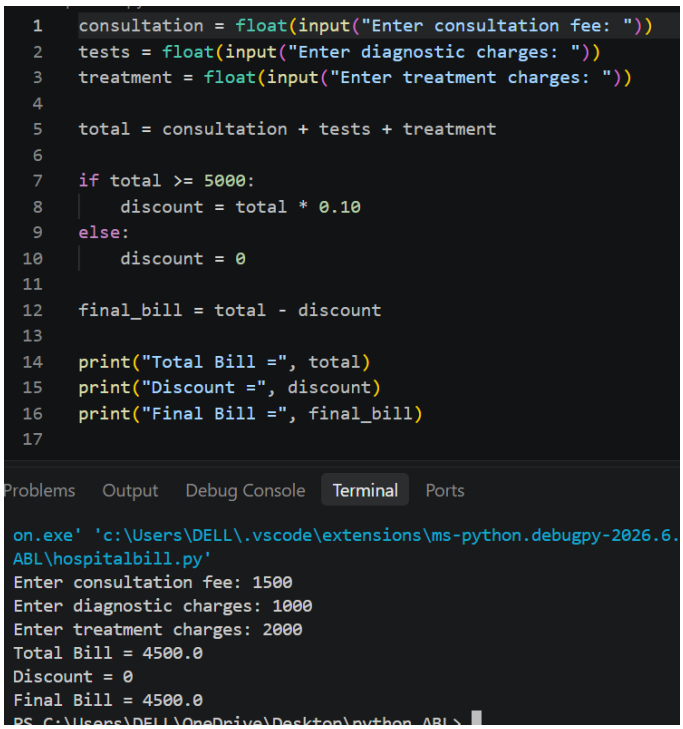
print("Total Bill =", total)
print("Discount =", discount)
print("Final Bill =", final_bill)
```

Case 3: Hospital Billing — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
1500, 1000, 2000	4500.0, 0, 4500.0	Total is below 5000, so no discount.
2000, 1500, 2000	4950.0, 0, 4950.0	Total is below 5000, so no discount.
2500, 1500, 1500	5500.0, 550.0, 4950.0	10% discount applies.

Output Screenshot



Conclusion

The program calculates hospital charges and applies the discount according to the total bill.

Case 4: Employee Payroll

Problem Statement

Calculate gross salary, deduction, and net salary using basic salary, allowance, and overtime amount.

Objectives

Perform salary calculations and use a condition to select the deduction rate.

Assumptions

Salary inputs are non-negative. A 10% deduction applies when gross salary is greater than 30000; otherwise, a 5% deduction applies.

Algorithm

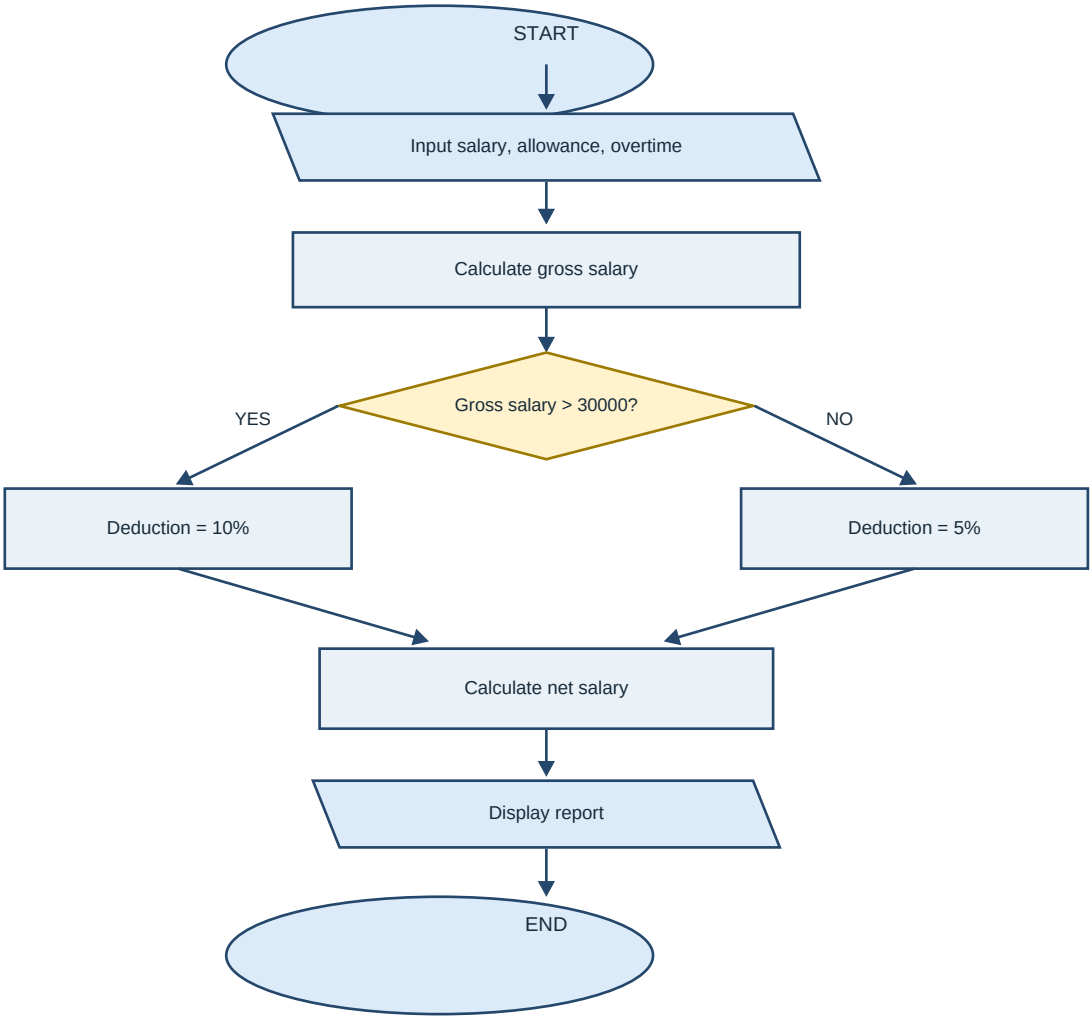
1. Start.
2. Read basic salary, allowance, and overtime amount.
3. Add them to calculate gross salary.
4. If gross salary is greater than 30000, calculate a 10% deduction; otherwise calculate a 5% deduction.
5. Subtract deduction from gross salary to calculate net salary.
6. Display gross salary, deduction, and net salary.
7. Stop.

Pseudocode

```
START
INPUT basic_salary, allowance, overtime
gross_salary = basic_salary + allowance + overtime
IF gross_salary > 30000
    deduction = gross_salary * 0.10
ELSE
    deduction = gross_salary * 0.05
END IF
net_salary = gross_salary - deduction
OUTPUT gross_salary, deduction, net_salary
STOP
```

Case 4: Employee Payroll — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Case 4: Employee Payroll — Python Source Code

```
basic_salary = float(input("Enter basic salary: "))
allowance = float(input("Enter allowance: "))
overtime = float(input("Enter overtime amount: "))

gross_salary = basic_salary + allowance + overtime

if gross_salary > 30000:
    deduction = gross_salary * 0.10
else:
    deduction = gross_salary * 0.05

net_salary = gross_salary - deduction

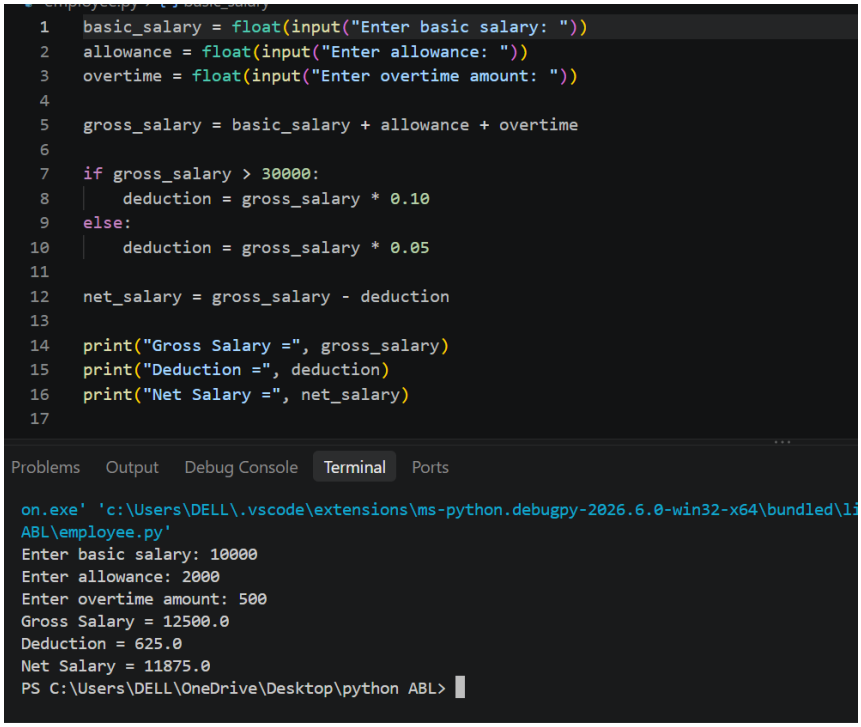
print("Gross Salary =", gross_salary)
print("Deduction =", deduction)
print("Net Salary =", net_salary)
```

Case 4: Employee Payroll — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
10000, 2000, 500	12500.0; 625.0; 11875.0	Gross salary is at most 30000, so 5% deduction applies.
25000, 4000, 2000	31000.0; 3100.0; 27900.0	Gross salary is greater than 30000, so 10% deduction applies.

Output Screenshot



Conclusion

The program calculates employee take-home salary by applying the correct deduction rate.

Case 7: Retail Billing

Problem Statement

Calculate the total cost of multiple products and apply a 10% discount when the total is at least 1000.

Objectives

Use a for loop to process products and calculate a bill from each product's price and quantity.

Assumptions

The number of products and quantities are positive whole numbers, prices are non-negative, and a 10% discount applies at a total of 1000 or more.

Algorithm

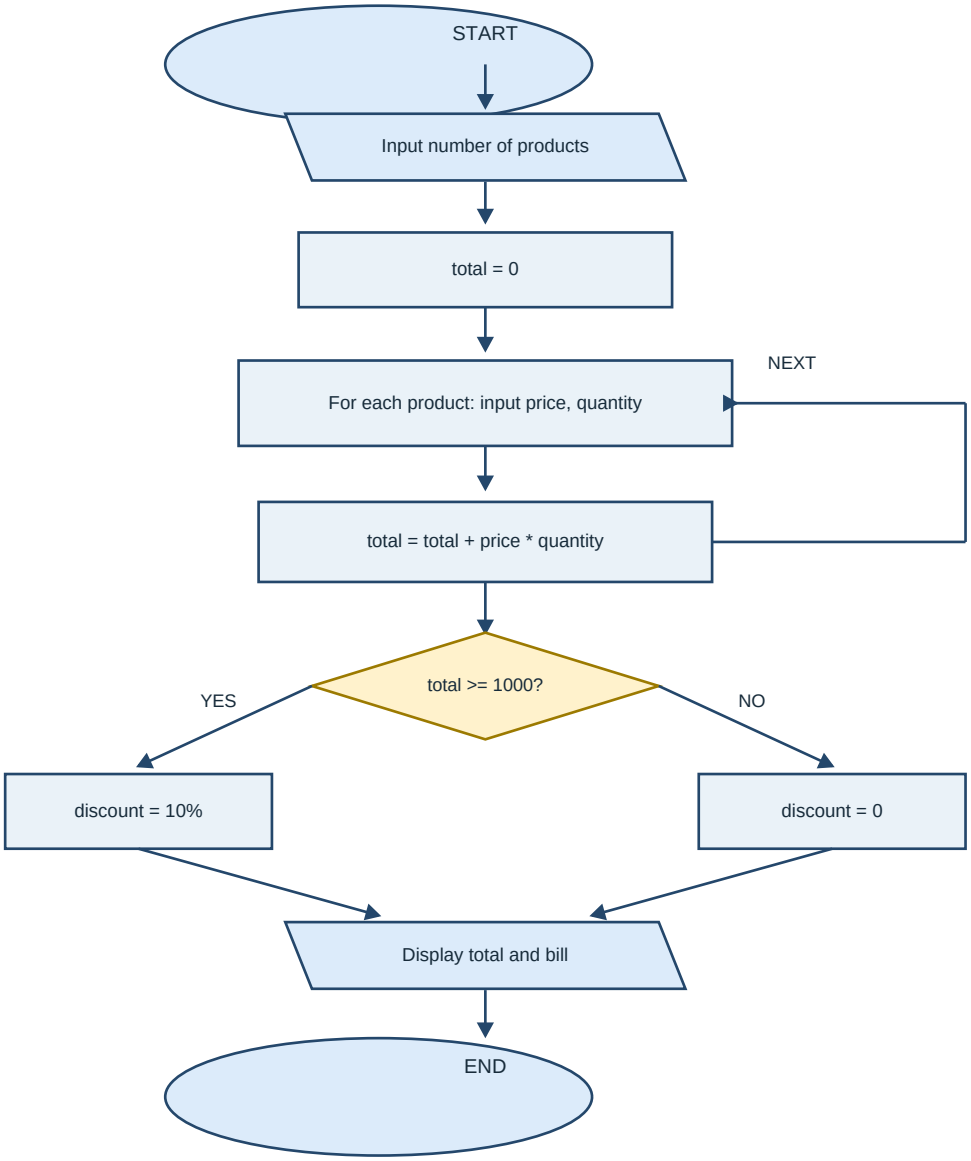
1. Start.
2. Set total to 0.
3. Read number of products.
4. Repeat for each product: read price and quantity, then add price × quantity to total.
5. If total is at least 1000, set discount to 10% of total; otherwise set discount to 0.
6. Calculate final bill as total minus discount.
7. Display total, discount, and final bill.
8. Stop.

Pseudocode

```
START
total = 0
INPUT n
FOR each product from 1 to n
    INPUT price, quantity
    total = total + price * quantity
END FOR
IF total >= 1000
    discount = total * 0.10
ELSE
    discount = 0
END IF
final_bill = total - discount
OUTPUT total, discount, final_bill
STOP
```

Case 7: Retail Billing — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Case 7: Retail Billing — Python Source Code

```
total = 0

n = int(input("Enter number of products: "))

for i in range(n):
    price = float(input("Enter product price: "))
    quantity = int(input("Enter quantity: "))
    total = total + price * quantity

if total >= 1000:
    discount = total * 0.10
else:
    discount = 0

final_bill = total - discount

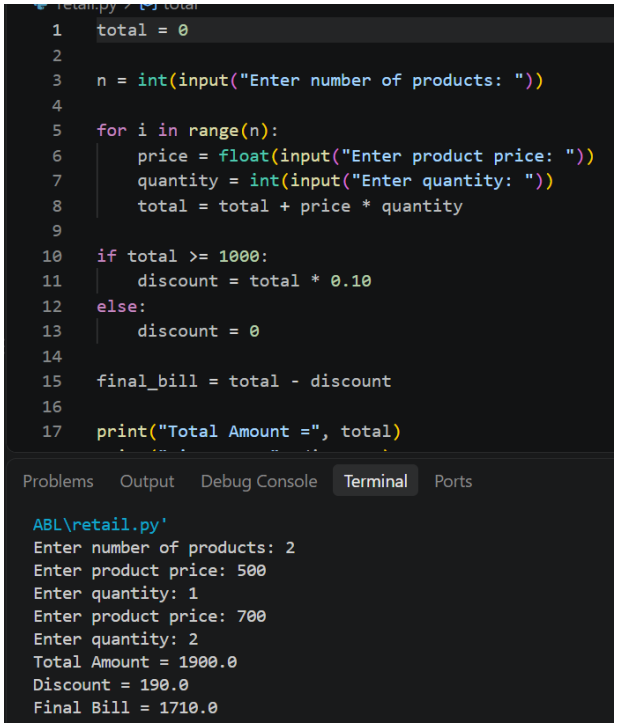
print("Total Amount =", total)
print("Discount =", discount)
print("Final Bill =", final_bill)
```

Case 7: Retail Billing — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
2 products: 500×1, 700×2	1900.0; 190.0; 1710.0	10% discount applies.
1 product: 400×1	400.0; 0; 400.0	No discount applies.

Output Screenshot



Conclusion

The program uses iteration to add product amounts and a condition to calculate the discount.

Case 8: Manufacturing Cost and Profit

Problem Statement

Calculate total manufacturing cost and determine whether the selling price produces profit, loss, or no profit and no loss.

Objectives

Add cost components, calculate profit, and classify the result using conditional statements.

Assumptions

All costs and selling price are entered as non-negative amounts. Profit is selling price minus total cost.

Algorithm

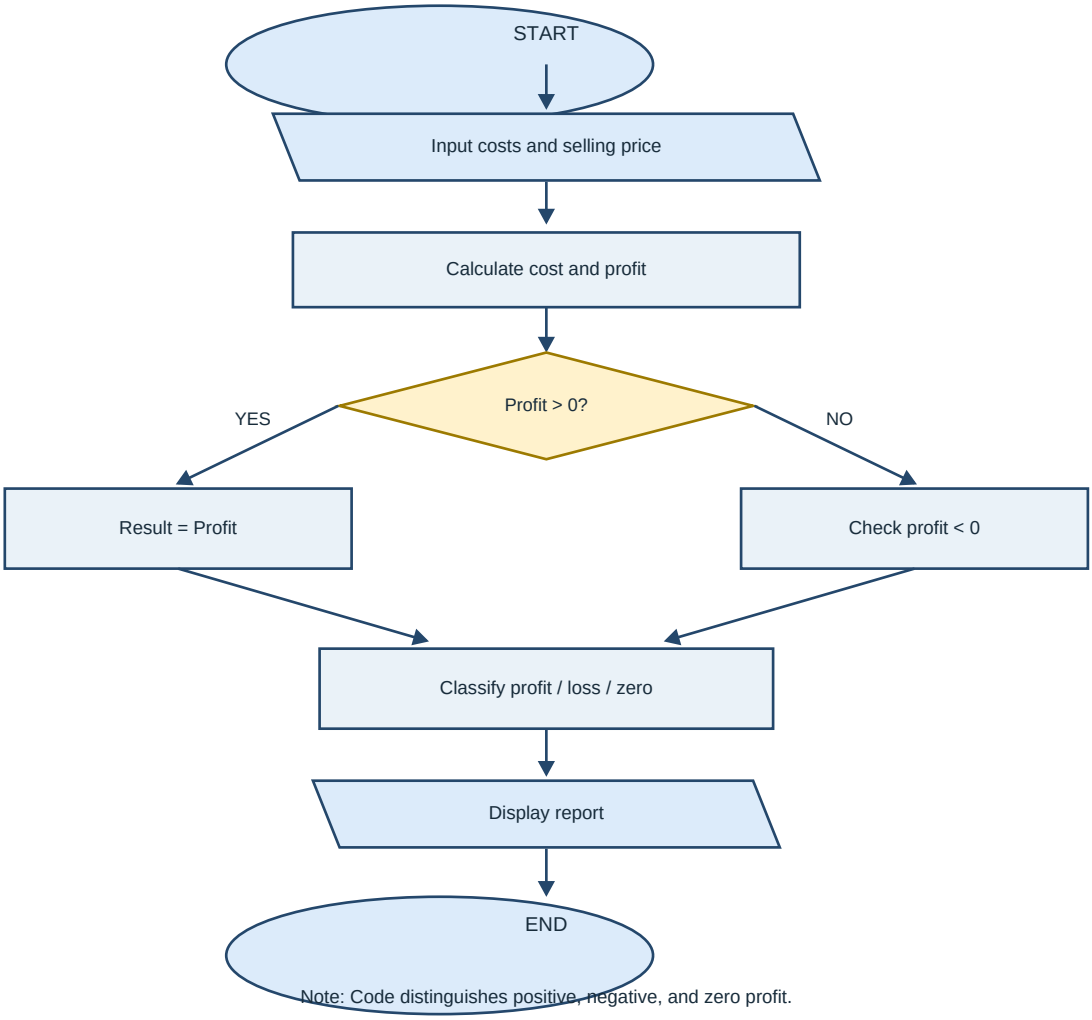
1. Start.
2. Read material cost, labour cost, machine cost, and selling price.
3. Add the three costs to calculate total cost.
4. Calculate profit as selling price minus total cost.
5. If profit is positive, display Profit; if negative, display Loss; otherwise display No Profit No Loss.
6. Display total cost and profit.
7. Stop.

Pseudocode

```
START
INPUT material_cost, labour_cost, machine_cost, selling_price
total_cost = material_cost + labour_cost + machine_cost
profit = selling_price - total_cost
IF profit > 0
    OUTPUT Profit
ELSE IF profit < 0
    OUTPUT Loss
ELSE
    OUTPUT No Profit No Loss
END IF
OUTPUT total_cost, profit
STOP
```

Case 8: Manufacturing Cost and Profit — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Note: Code distinguishes positive, negative, and zero profit.

Case 8: Manufacturing Cost and Profit — Python Source Code

```
material_cost = float(input("Enter material cost: "))
labour_cost = float(input("Enter labour cost: "))
machine_cost = float(input("Enter machine cost: "))
selling_price = float(input("Enter selling price: "))

total_cost = material_cost + labour_cost + machine_cost
profit = selling_price - total_cost

print("Total Cost =", total_cost)
print("Profit =", profit)

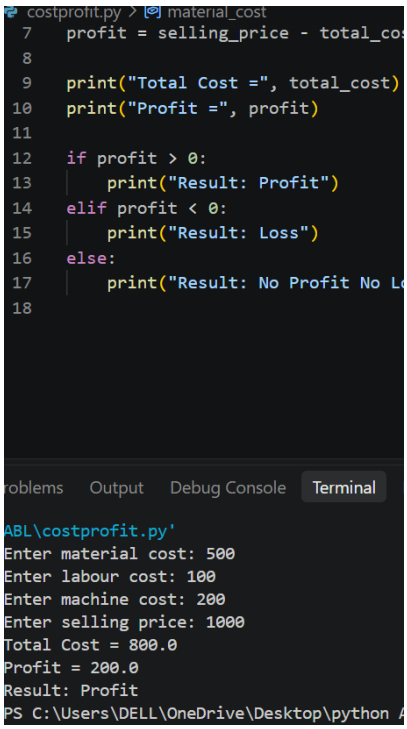
if profit > 0:
    print("Result: Profit")
elif profit < 0:
    print("Result: Loss")
else:
    print("Result: No Profit No Loss")
```

Case 8: Manufacturing Cost and Profit — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
500, 100, 200, 1000	Total cost 800.0; profit 200.0; Profit	Selling price exceeds total cost.
500, 100, 200, 700	Total cost 800.0; profit -100.0; Loss	Selling price is below total cost.
500, 100, 200, 800	Total cost 800.0; profit 0.0; No Profit No Loss	Selling price equals total cost.

Output Screenshot



Conclusion

The program identifies the financial result by comparing selling price with total cost.

Case 11: Examination Result

Problem Statement

Calculate total and average marks for three subjects and determine whether the student passes all subjects.

Objectives

Calculate total and average marks and apply a combined condition to check the pass requirement.

Assumptions

Each subject mark is between 0 and 100. A student passes only if all three subject marks are at least 35.

Algorithm

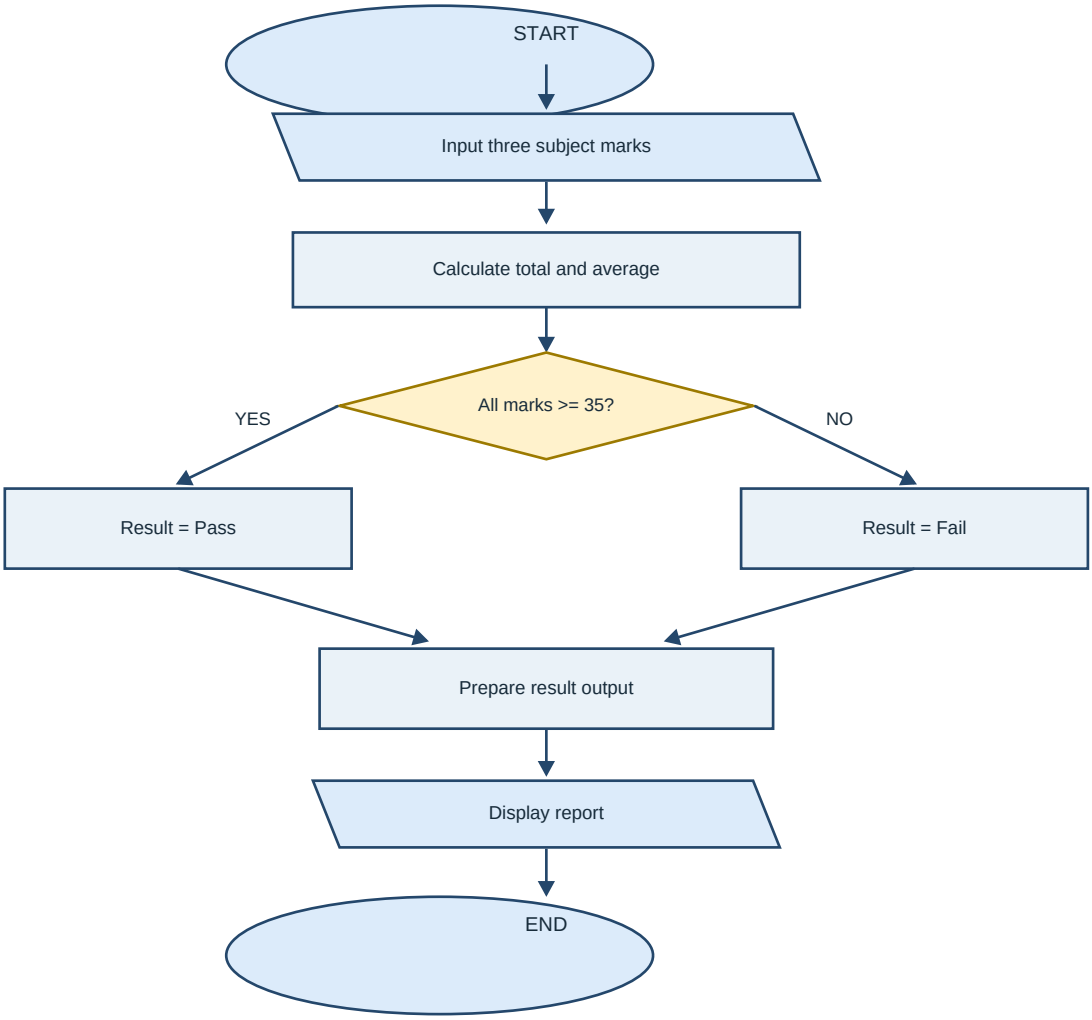
1. Start.
2. Read marks for three subjects.
3. Calculate total and average marks.
4. If all three marks are at least 35, display Pass; otherwise display Fail.
5. Display total and average marks.
6. Stop.

Pseudocode

```
START
INPUT m1, m2, m3
total = m1 + m2 + m3
average = total / 3
IF m1 >= 35 AND m2 >= 35 AND m3 >= 35
    OUTPUT Pass
ELSE
    OUTPUT Fail
END IF
OUTPUT total, average
STOP
```

Case 11: Examination Result — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Case 11: Examination Result — Python Source Code

```
m1 = int(input("Enter marks in Subject 1: "))
m2 = int(input("Enter marks in Subject 2: "))
m3 = int(input("Enter marks in Subject 3: "))

total = m1 + m2 + m3
average = total / 3

print("Total Marks =", total)
print("Average =", average)

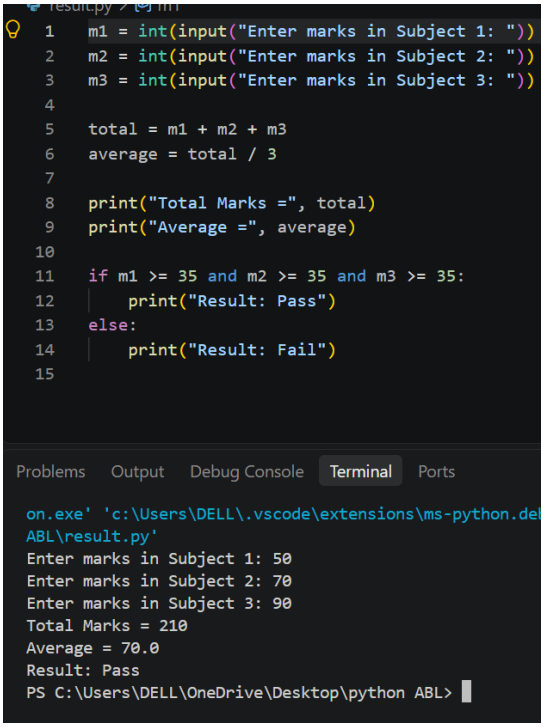
if m1 >= 35 and m2 >= 35 and m3 >= 35:
    print("Result: Pass")
else:
    print("Result: Fail")
```

Case 11: Examination Result — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
50, 70, 90	Total 210; average 70.0; Pass	All subject marks are at least 35.
50, 30, 90	Total 170; average 56.67; Fail	One subject mark is below 35.

Output Screenshot



Conclusion

The program calculates academic totals and checks that every subject meets the minimum passing mark.

Case 12: Attendance Recovery

Problem Statement

Find how many additional classes a student must attend to reach 75% attendance.

Objectives

Use a while loop to update attended and total classes until attendance reaches the required percentage.

Assumptions

Total classes must be greater than zero and attended classes must be non-negative and no greater than total classes. The required attendance is 75%.

Algorithm

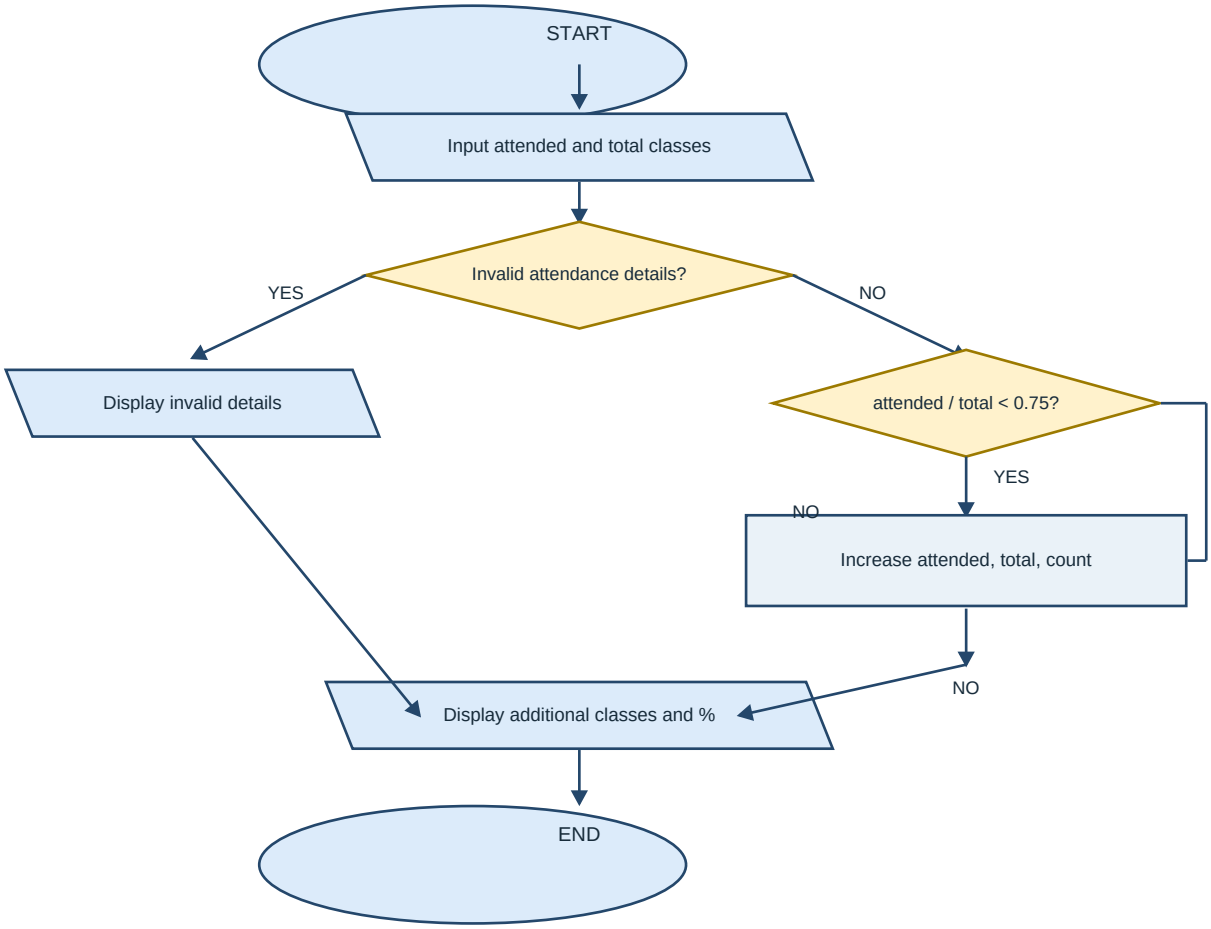
1. Start.
2. Read classes attended and total classes.
3. If attended is negative or total is less than attended, display invalid details.
4. Otherwise, set count to 0.
5. While attended / total is less than 0.75, increase attended, total, and count by 1.
6. Display additional classes required and final attendance percentage.
7. Stop.

Pseudocode

```
START
INPUT attended, total
IF attended < 0 OR total < attended
    OUTPUT Invalid attendance details
ELSE
    count = 0
    WHILE attended / total < 0.75
        attended = attended + 1
        total = total + 1
        count = count + 1
    END WHILE
    OUTPUT count, attended / total * 100
END IF
STOP
```

Case 12: Attendance Recovery — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Case 12: Attendance Recovery — Python Source Code

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes: "))

if attended < 0 or total < attended:
    print("Invalid attendance details")
else:
    count = 0

    while attended / total < 0.75:
        attended = attended + 1
        total = total + 1
        count = count + 1

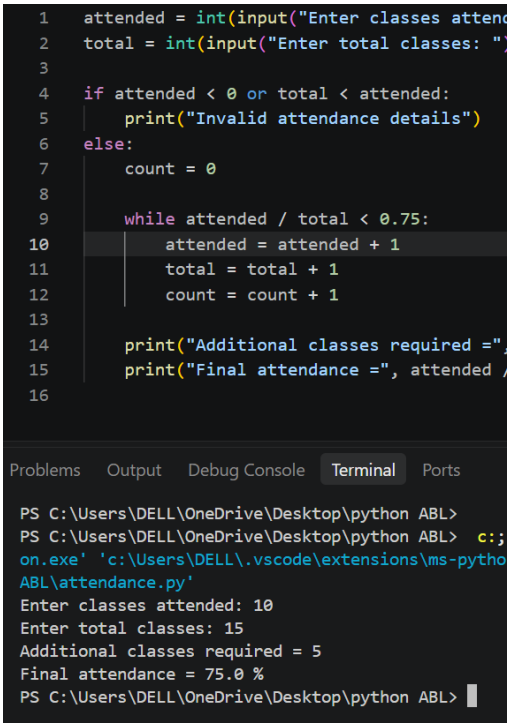
    print("Additional classes required =", count)
    print("Final attendance =", attended / total * 100, "%")
```

Case 12: Attendance Recovery — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
10 attended, 15 total	5 additional classes; final attendance 75.0%	The loop runs until the ratio reaches 0.75.
8 attended, 10 total	0 additional classes; final attendance 80.0%	Attendance already exceeds 75%.

Output Screenshot



Conclusion

The program uses a while loop to determine the minimum additional classes needed to reach 75% attendance.

Case 13: ATM Transaction Simulation

Problem Statement

Simulate an ATM with deposit, withdrawal, balance enquiry, and exit options.

Objectives

Use a menu-driven while loop, conditions, and balance updates to handle simple ATM transactions.

Assumptions

The starting balance is 5000. Deposits and withdrawals must be positive; withdrawals cannot exceed the current balance.

Algorithm

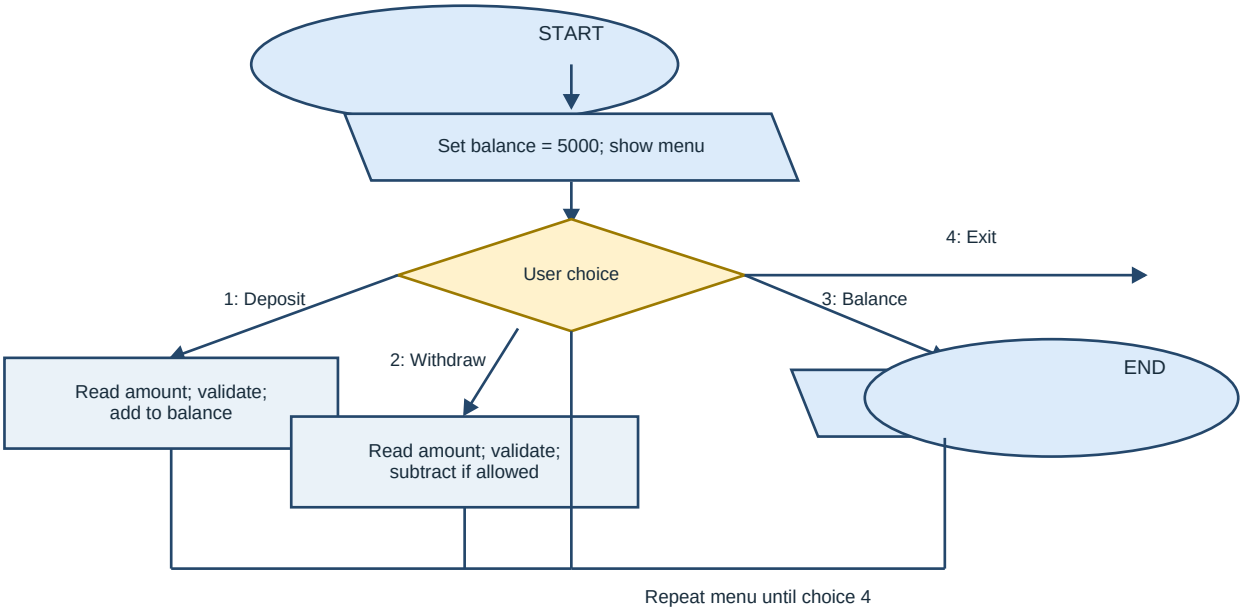
1. Start with balance = 5000.
2. Display the ATM menu and read the user's choice.
3. If choice is Deposit, read amount and add it only if positive.
4. If choice is Withdraw, read amount and subtract it only if positive and no greater than balance.
5. If choice is Check Balance, display balance.
6. If choice is Exit, display a thank-you message and stop the loop.
7. For any other choice, display Invalid choice.
8. Repeat the menu until Exit is selected.

Pseudocode

```
START
balance = 5000
WHILE True
    DISPLAY menu
    INPUT choice
    IF choice == 1
        INPUT amount
        IF amount > 0: balance = balance + amount
        ELSE: OUTPUT Invalid amount
    ELSE IF choice == 2
        INPUT amount
        IF amount > 0 AND amount <= balance: balance = balance - amount
        ELSE: OUTPUT Invalid amount or insufficient balance
    ELSE IF choice == 3
        OUTPUT balance
    ELSE IF choice == 4
        OUTPUT Thank you; EXIT LOOP
    ELSE
        OUTPUT Invalid choice
    END IF
END WHILE
STOP
```

Case 13: ATM Transaction Simulation — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Case 13: ATM Transaction Simulation — Python Source Code

```
balance = 5000

while True:
    print("\\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

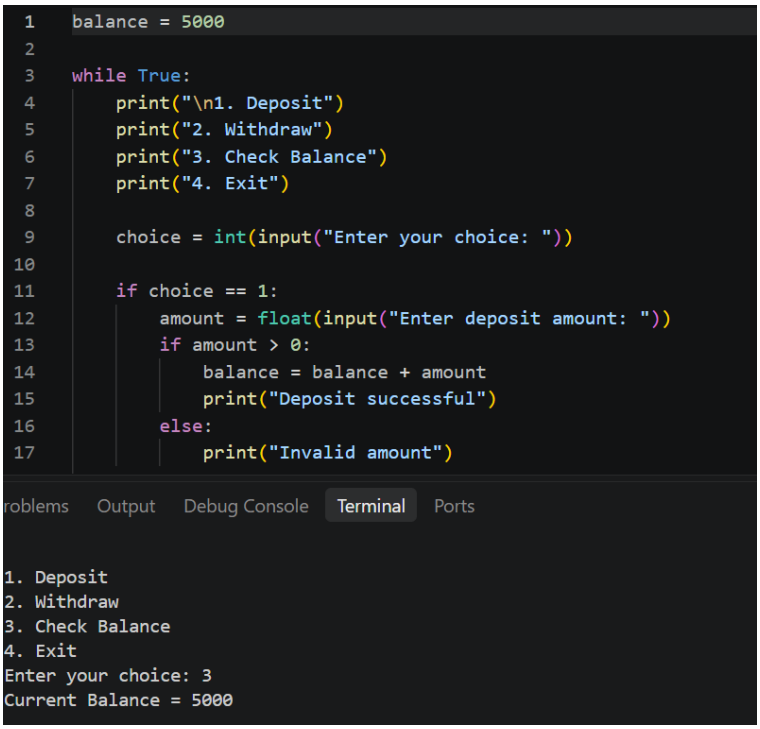
    if choice == 1:
        amount = float(input("Enter deposit amount: "))
        if amount > 0:
            balance = balance + amount
            print("Deposit successful")
        else:
            print("Invalid amount")
    elif choice == 2:
        amount = float(input("Enter withdrawal amount: "))
        if amount > 0 and amount <= balance:
            balance = balance - amount
            print("Withdrawal successful")
        else:
            print("Invalid amount or insufficient balance")
    elif choice == 3:
        print("Current Balance =", balance)
    elif choice == 4:
        print("Thank you for using ATM")
        break
    else:
        print("Invalid choice")
```

Case 13: ATM Transaction Simulation — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
Choice 3	Current Balance = 5000	Displays initial balance.
Choice 1; amount 1000, then choice 3	Deposit successful; Current Balance = 6000	Valid deposit increases balance.
Choice 4	Thank you for using ATM	Exit ends the loop.

Output Screenshot



Conclusion

The program demonstrates a menu-driven transaction system with input validation and repeated execution until the user exits.

Case 27: Numeric Pattern Generator

Problem Statement

Print a number pattern in which each row displays numbers from 1 up to the row number.

Objectives

Use nested for loops to generate a simple numeric pattern and validate the number of rows.

Assumptions

The number of rows should be a positive integer.

Algorithm

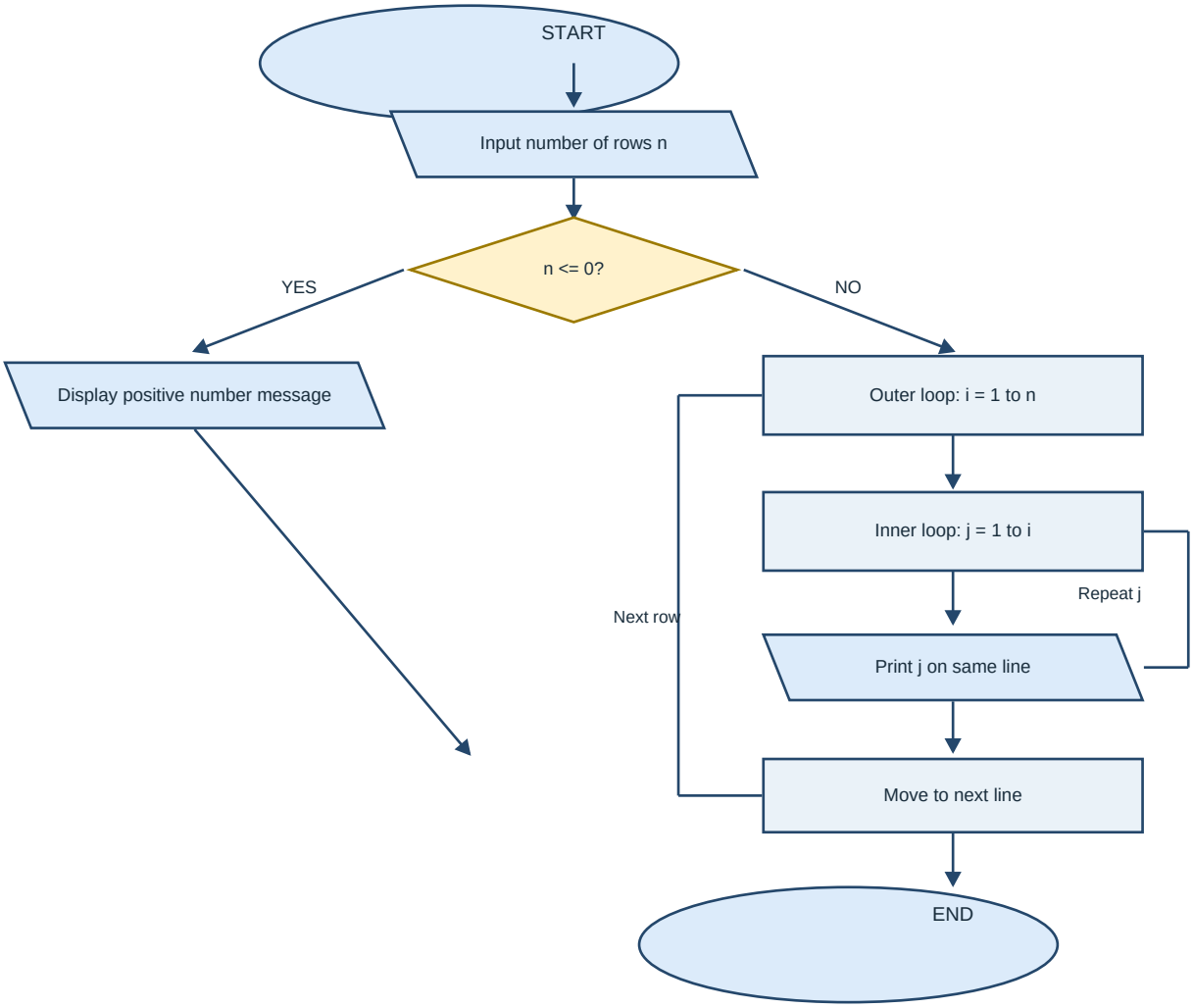
1. Start.
2. Read the number of rows n.
3. If n is at most 0, display a message requesting a positive number.
4. Otherwise, repeat i from 1 to n.
5. For each row i, print j from 1 to i on the same line.
6. Move to the next line after each row.
7. Stop.

Pseudocode

```
START
INPUT n
IF n <= 0
    OUTPUT Enter a positive number
ELSE
    FOR i = 1 TO n
        FOR j = 1 TO i
            OUTPUT j on same line
        END FOR
        MOVE to next line
    END FOR
END IF
STOP
```

Case 27: Numeric Pattern Generator — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Case 27: Numeric Pattern Generator — Python Source Code

```
n = int(input("Enter number of rows: "))

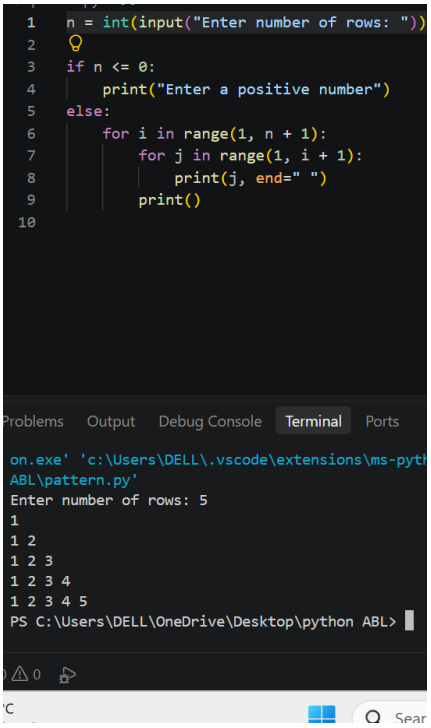
if n <= 0:
    print("Enter a positive number")
else:
    for i in range(1, n + 1):
        for j in range(1, i + 1):
            print(j, end=" ")
        print()
```

Case 27: Numeric Pattern Generator — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
n = 5	1 / 1 2 / 1 2 3 / 1 2 3 4 / 1 2 3 4 5	Five rows are printed.
n = 0	Enter a positive number	Non-positive row count is rejected.

Output Screenshot



Conclusion

The program uses nested loops to print a structured numeric pattern.

Case 39: Multi-Stage Examination Eligibility System

Problem Statement

Check examination eligibility using attendance, laboratory completion, internal marks, and fee payment.

Objectives

Evaluate multiple requirements, count unmet requirements, and display the final eligibility status.

Assumptions

Minimum attendance is 75%, minimum internal marks are 40, laboratory work must be completed, and all fees must be paid. Answers for laboratory completion and fees are yes/no.

Algorithm

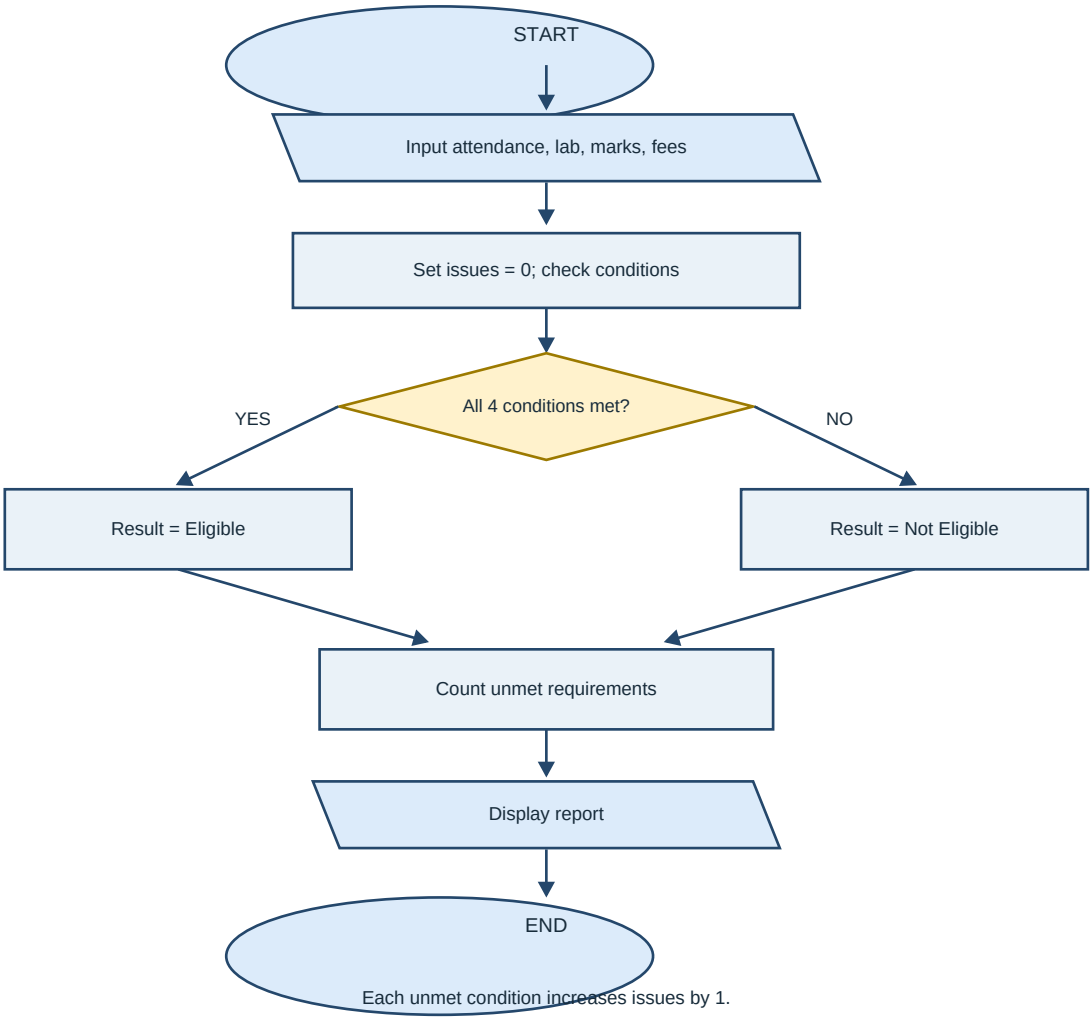
1. Start.
2. Read attendance, lab completion, internal marks, and fee payment.
3. Set issues to 0 and print the report heading.
4. If attendance is below 75%, report the issue and increment issues.
5. If lab completion is not yes, report the issue and increment issues.
6. If internal marks are below 40, report the issue and increment issues.
7. If fees are not paid, report the issue and increment issues.
8. If issues equals 0, display Eligible; otherwise display Not Eligible.
9. Display total unmet requirements and stop.

Pseudocode

```
START
INPUT attendance, lab, internal_marks, fees
issues = 0
IF attendance < 75: report low attendance; issues = issues + 1
IF lower(lab) != yes: report incomplete lab; issues = issues + 1
IF internal_marks < 40: report low marks; issues = issues + 1
IF lower(fees) != yes: report unpaid fees; issues = issues + 1
IF issues == 0
    OUTPUT Eligible for examination
ELSE
    OUTPUT Not eligible for examination
END IF
OUTPUT issues
STOP
```

Case 39: Multi-Stage Examination Eligibility System — Standard Flowchart

Oval: Start/End | Parallelogram: Input/Output | Rectangle: Process | Diamond: Decision



Case 39: Multi-Stage Examination Eligibility System — Python Source Code

```
attendance = float(input("Enter attendance percentage: "))
lab = input("Are laboratory works completed? (yes/no): ")
internal_marks = int(input("Enter internal marks: "))
fees = input("Are all fees paid? (yes/no): ")

issues = 0

print("\\n--- Examination Eligibility Report ---")

if attendance < 75:
    print("Requirement not met: Low attendance")
    issues = issues + 1

if lab.lower() != "yes":
    print("Requirement not met: Laboratory work incomplete")
    issues = issues + 1

if internal_marks < 40:
    print("Requirement not met: Internal marks are low")
    issues = issues + 1

if fees.lower() != "yes":
    print("Requirement not met: Fees not paid")
    issues = issues + 1

if issues == 0:
    print("Result: Eligible for examination")
else:
    print("Result: Not eligible for examination")

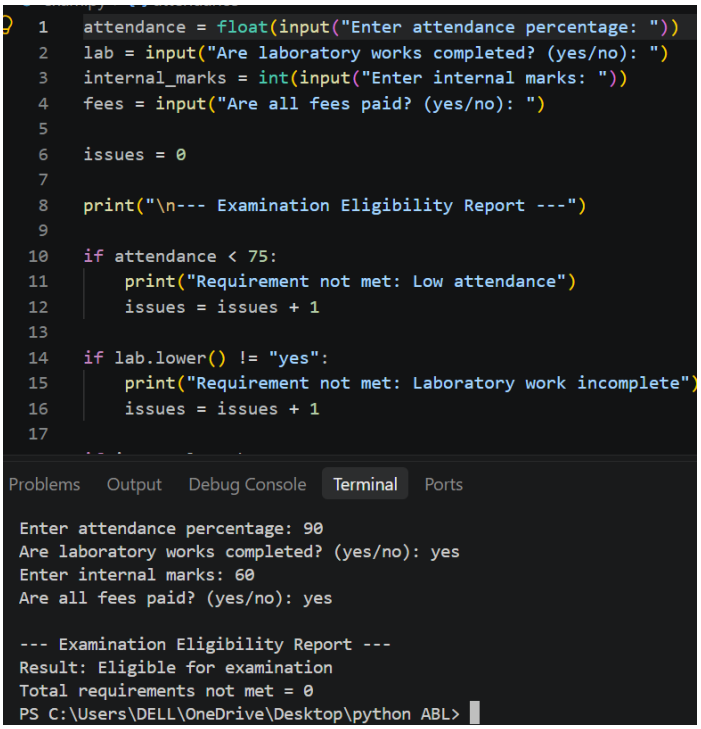
print("Total requirements not met =", issues)
```

Case 39: Multi-Stage Examination Eligibility System — Testing and Output

Test Cases

Input	Expected / observed result	Purpose
90, yes, 60, yes	Eligible for examination; issues = 0	All requirements are satisfied.
60, no, 30, no	Not eligible; issues = 4	All four requirements are unmet.

Output Screenshot



Conclusion

The program checks four separate eligibility requirements and reports the total number of unmet conditions.